NORTHERN UNIVERSITY OF BUSINESS AND TECHNOLOGY KHULNA

COURSE CODE: CSE 4204

COURSE TITLE: Mobile Computing Lab

[Project Name: AI Expense Analyzer]

Submitted by –	**Submitted to –**
Team: CSE4204-8A-T04 Naim Sheikh ID: 11210320634 Ayon Banerjee ID: 11220120820 Md Shakinul Islam ID: 11220120822 Afia Rahman ID: 11220320870	Md. Riaz Mahmud Assistant Professor Department of Computer Science and Engineering Northern University of Business and Technology Khulna

Date of Submission – 15/06/2026

Index

AI Expense Analyzer

1. Team Information:

Team Name: CSE4204-8A-T04

Section: 8A

Project Title: AI Expense Analyzer

Team Leader: Md. Shakinul Islam

GitHub Repository:https://github.com/shakinul-islam/AI-Expense-Analyzer

1.1 Team Information:

Serial	Name	Student ID	Technology	Responsibility
1	Shakinul Islam	11220120822	MongoDB	Database Management & Testing
2	Naim Sheikh	11210320634	Flutter	Frontend Development & UI Design
3	Ayon Banerjee	11220120820	Node.js + Express.js	Backend Development
4	Afia Rahman	11220320870	Gemini AP	AI Integration

2. Project Information:

2.1 Overview:

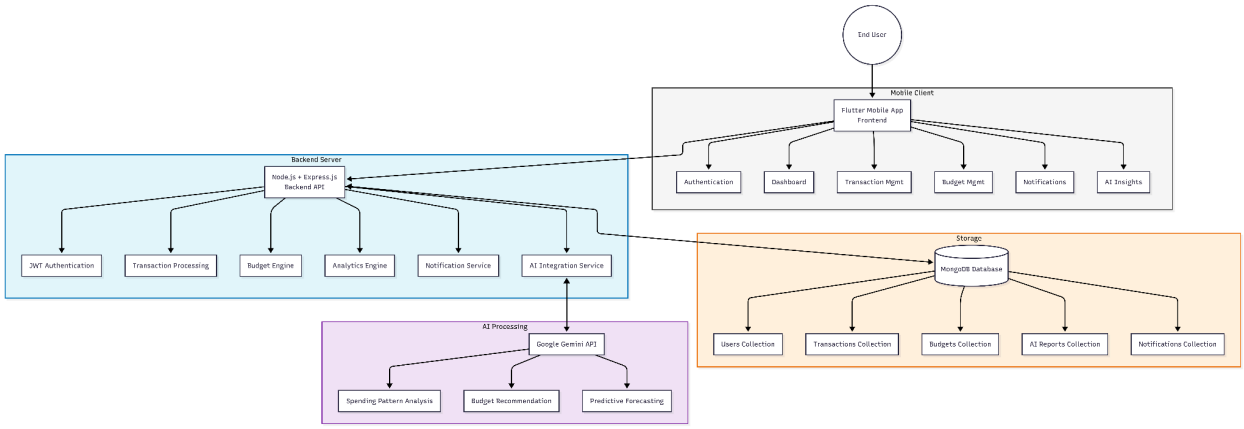
The AI Expense Analyzer is a cross-platform intelligent asset ecosystem utilizing a Flutter frontend client, managed by an asynchronous Node.js backend layered over Express.js middleware, and persistent within a MongoDB document database. To upgrade past standard, static digital ledger interfaces that require manual evaluation, the environment integrates the advanced Google Gemini API. This enables automated transaction pattern tagging, real-time localized optimization advice, predictive budgeting models, and algorithmic waste tracking to instill long-term fiscal discipline.

2.2 Project Information:

Title	**Description**
Project Name	AI Expense Analyzer
Problem	Users cannot understand their spending patterns from raw transaction data.
Solution	AI-powered financial insights, automated categorization, predictive budgeting.
Target Users	Individual consumers who want to save money and track expenses.
Platform	Cross-platform mobile (Android + iOS)

3. Engineering Diagrams & Visual Models

3.1 Architecture Diagram:



3.2 Architecture Component Breakdown:

3.2.1 Frontend Layer (Flutter):

Component	Technology	Functionality
Login Screen	Flutter (Dart)	User authentication UI, email/password input
Dashboard Screen	Flutter (Dart)	Spending summary, charts, key metrics display
Add Expense Screen	Flutter (Dart)	Transaction entry form with amount, category, date
AI Insights Screen	Flutter (Dart)	Display AI-generated financial advice cards
Profile Screen	Flutter (Dart)	User settings, currency preference, savings goal
Notification Panel	Flutter (Dart)	Show budget alerts and system notifications

3.2.2 Backend Layer (Node.js + Express.js):

Module	Functionality
Authentication	User registration, login, JWT token management.
Dashboard	Fetch and aggregate user spending summary.
Transaction	CRUD operations for income/expense entries.
Budget	Set and track monthly budget limits.
Notifications	Generate and send budget alerts.
AI Insights	Proxy requests to Gemini API.

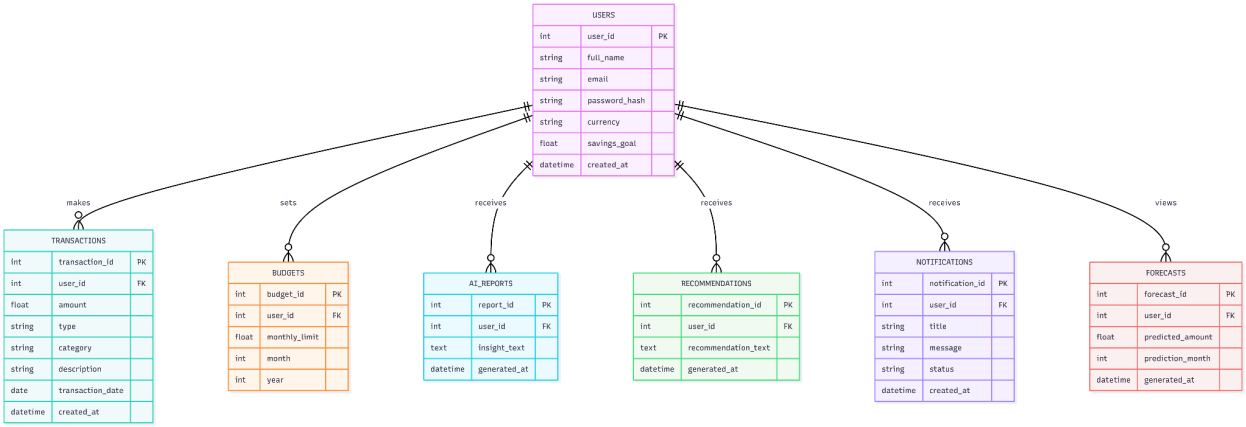
3.2.3 Storage Layer (MongoDB):

Collection	Purpose
Users Collection	Store user profiles and credentials
Transactions Collection	Store all income/expense entries
Budgets Collection	Store monthly budget limits
AI Reports Collection	Cache AI-generated insights
Notifications Collection	Store notification history

3.2.4 AI Processing Layer (Google Gemini API):

Service	Function
Spending Pattern Analysis	Identify wasteful spending categories.
Budget Recommendation	Suggest personalized category limits.
Predictive Forecasting	Predict next month's expenses.

4. ER Diagram (Database Design):



4.1 Complete Schema Definition:

According to diagram,

Table 1: Users

Column	Type	Constraint	Description
user_id	INT	PRIMARY KEY	Unique user identifier
full_name	STRING	NOT NULL	User's full name
email	STRING	UNIQUE, NOT NULL	Login email
password_hash	STRING	NOT NULL	bcrypt hashed password
currency	STRING	DEFAULT 'BDT'	Currency preference
savings_goal	FLOAT	DEFAULT 0	Monthly savings target
created_at	DATETIME	AUTO	Account creation timestamp

Table 2: transactions

Column	Type	Constraint	Description
id	INT	PRIMARY KEY	Unique transaction ID
user_id	INT	FOREIGN KEY → users	References user
amount	FLOAT	NOT NULL	Positive=income, Negative=expense
type	STRING	'income'/'expense'	Transaction direction
category	STRING	NOT NULL	Food, Transport, Shopping, etc.
description	STRING	—	User-entered text
transaction_date	DATE	NOT NULL	When money was spent
created_at	DATETIME	AUTO	Entry creation timestamp

Table 3: budgets

Column	Type	Constraint	Description
budget_id	INT	PRIMARY KEY	Unique budget ID
user_id	INT	FOREIGN KEY → users	References user
monthly_limit	FLOAT	NOT NULL	Total budget limit
month	INT	1-12	Month of budget
year	INT	NOT NULL	Year of budget
created_at	DATETIME	AUTO	Creation timestamp

Table 4: ai_reports

Column	Type	Constraint	Description
report_id	INT	PRIMARY KEY	Unique report ID
user_id	INT	FOREIGN KEY → users	References user
insight_text	TEXT	NOT NULL	AI-generated advice
generated _ at	DATETIME	AUTO	Generation timestamp

Table 5: recommendations

Column	Type	Constraint	Description
recomendation_id	INT	PRIMARY KEY	Unique ID
user_id	INT	FOREIGN KEY → users	References user
title	STRING	NOT NULL	Recommendation title
recomendation_text	TEXT	NOT NULL	Detailed advice
generated _ at	DATETIME	AUTO	Timestamp

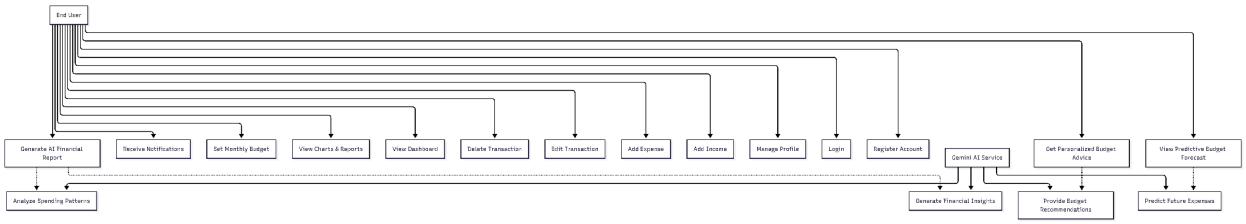
Table 6: notifications

Column	Type	Constraint	Description
notification_id	INT	PRIMARY KEY	Unique ID
user_id	INT	FOREIGN KEY → users	References user
title	STRING	NOT NULL	Notification title
message	TEXT	NOT NULL	Content
generated _ at	DATETIME	AUTO	Timestamp

Table 7: forecasts

Column	Type	Constraint	Description
forecast_id	INT	PRIMARY KEY	Unique ID
user_id	INT	FOREIGN KEY → users	References user
predicted_amount	FLOAT	NOT NULL	Predicted expense
predicted_month	INT	1-12	Month being predicted
generated _ at	DATETIME	AUTO	Timestamp

5. Use Case Diagram:



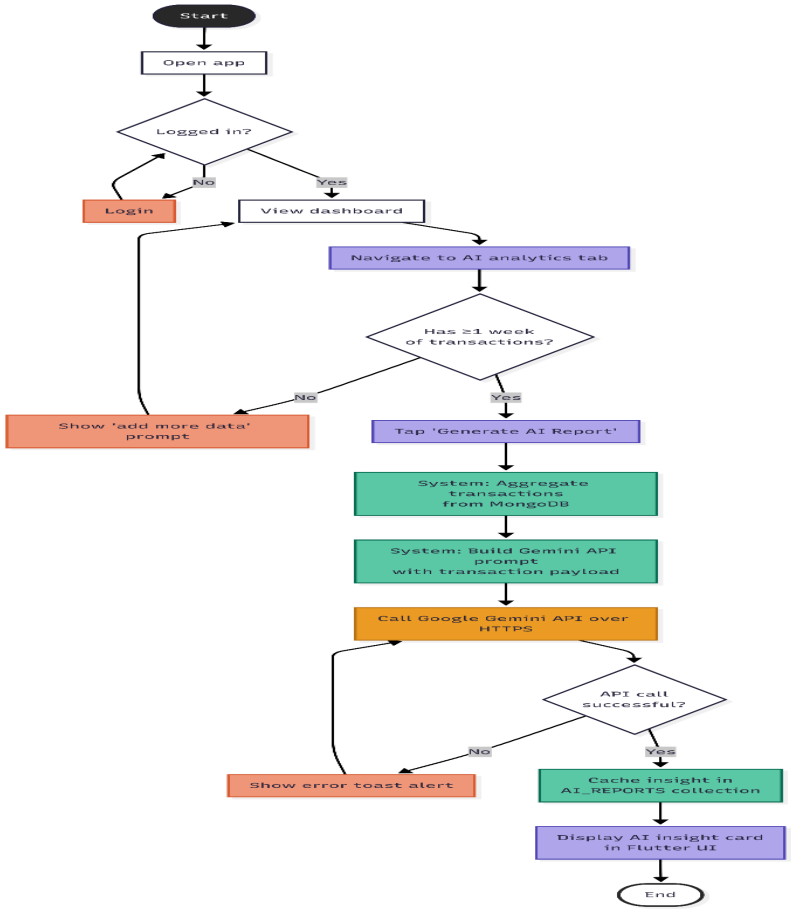
5.1 Use Case Descriptions:

Use Case	Actor	Description	AI Involved
View Dashboard	End User	View spending summary and charts.	❌
Set Monthly Budget	End User	Define budget limits.	❌
View Charts & Reports	End User	See visual analytics.	❌
Generate AI Financial Report	End User	Request AI spending analysis.	✅
Receive Notifications	End User	Get budget alerts.	✅

5.2 Detailed Use Case: Generate AI Financial Report

Element	Description
Actor	End User
Precondition	User has ≥7 days of transaction history.
Postcondition	AI advice displayed and cached
Main Flow	1. Navigate to AI tab → 2. Click Generate → 3. View advice
Exception	No transactions → Show "Add more data" prompt.

6. Activity Diagram:



6.1 Step-by-Step Process:

Step	Action	Decision	System Response
1	Open app	–	Launch Flutter
2	Check login	Logged in?	If No → Login screen
3	View dashboard	–	Main UI
4	Go to AI Analytics tab	–	AI screen
5	Check transactions	Has ≥1 week?	If No →Add data
6	Tap "Generate AI Report"	–	Loading
7	Aggregate transactions	–	Query MongoDB
8	Build Gemini prompt	–	Construct prompt
9	Call Gemini API	–	HTTPS request
10	Check API	Success?	If No → Error toast
11	Cache insight	–	Save to MongoDB
12	Display AI card	–	Show advice
13	End	–	User reads advice

7. API Design:

7.1 Authentication APIs:

Endpoint	Method	Purpose	Input	Output
/api/auth/register	POST	Create new user	{ full_name, email, password, currency, savings_goal }	{ message, user_id}
/api/auth/login	POST	User login	{ email, password }	{ token,user_id, full_name }
/api/auth/logout	POST	Invalidate JWT	Authorization: Bearer <token>	{ message }

7.2 User Profile APIs:

Endpoint	Method	Purpose	Input	Output
/api/user/profile	GET	View profile	Authorization: Bearer <token>	{ token,user_id, full_name }
/api/user/profile	PUT	Update profile	{ full_name, password, currency, savings_goal }	{ message }

7.3 Transaction APIs:

Endpoint	Method	Purpose	Input	Output
/api/transactions	POST	Add transaction	{ amount, type, description, date }	{transaction_id, category, message }
/api/transactions	GET	View all	Query: ?month, ?year, ?type	[{transaction_id, amount }]
/api/transactions/{id}	PUT	Edit transaction	{ amount, type, category, description, date }	{ message }
/api/transactions/{id}	DELETE	Delete transaction	Authorization: Bearer <token>	{ message }

7.4 Budget APIs:

Endpoint	Method	Purpose	Input	Output
/api/budget	POST	Set budget	{ monthly_limit, month, year }	{ budget_id, message }
/api/budget	GET	View budget	Query: ?month, ?year	{ budget_id, monthly_limit, spent_amont }

7.5 AI APIs (Gemini Integration):

Endpoint	Method	Purpose	Input	Output
/api/ai/report	POST	Generate insights	{ start_date, end_date }	{ report_id, insight_text, generated_ at }
/api/ai/report/history	GET	View history	Authorization: Bearer <token>	[{ report_id, insight_text }]
/api/ai/recommendation	POST	Budget advice	{ month, year }	{recommendation_id, recommendation_ text }
/api/ai/forecast	GET	Predictive forecast	Query: ?month, ?year	{ forecast_id, predicted_amount }

7.6 Notification APIs:

Endpoint	Method	Purpose	Input	Output
/api/notifications	GET	View notifications	Authorization: Bearer <token>	[{ notification_id, title }
/api/notifications /{id}/read	PUT	Mark as read	Authorization: Bearer <token>	{ message }

7.7 Dashboard APIs:

Endpoint	Method	Purpose	Input	Output
/api/dashboard	GET	Summary data	Query: ?month, ?year	{ total_income, total_expense }

8. AI Workflow:

8.1 Purpose of AI Integration

Traditional expense tracking systems primarily store and display financial data. However, users often struggle to identify unnecessary spending patterns, optimize budgets, or forecast future expenses from raw transaction records. To address this limitation, Google Gemini AI is integrated into the system to transform transaction data into actionable financial intelligence, enabling data-driven decision-making and personalized financial guidance.

8.2 Selected AI Service

The system utilizes Google Gemini API (gemini-1.5-flash model) for AI-powered financial analysis.

- Fast response time (typically under 7 seconds, aligned with NFR-01)
- Cost-effective text generation
- Strong reasoning and analytical capabilities
- Scalable cloud-based architecture
- Easy integration with Node.js backend services

8.3 AI-Driven Features and Workflow

Feature 1: Spending Insight (FR-05)

Workflow:

1. User requests an AI-generated spending report.
2. Backend retrieves the last 30 days of transaction data from MongoDB.
3. A structured prompt is generated containing transaction information.
4. The prompt is sent to Gemini API (gemini-1.5-flash).
5. Gemini analyzes spending behavior and identifies overspending patterns, unusual expenses, and potential waste areas.
6. The generated report is stored in the AI_REPORTS collection.
7. The insight is displayed as an interactive report card in the Flutter application.

Feature 2: Budget Recommendation (FR-06)

Workflow:

1. The system compares current spending against the user's monthly budget.
2. A personalized prompt is generated containing budget and expenditure information.
3. The prompt is submitted to Gemini API.

4. Gemini generates personalized saving recommendations and budget optimization strategies.
5. Recommendations are stored in the RECOMMENDATIONS collection.
6. Advice is delivered through notifications or recommendation cards within the application.

Feature 3: Predictive Forecast (FR-09)

Workflow:

1. The system retrieves transaction history from the previous three months.
2. Historical spending data is structured into a forecasting prompt.
3. The prompt is sent to Gemini API.
4. Gemini predicts future spending by category and estimates next month's total expenses.
5. Forecast results are stored in the FORECASTS collection.
6. Predicted values are visualized through charts and dashboard widgets in Flutter.

8.4 Input and Output Summary

Feature	Input to Gemini	Output from Gemini
Spending Insight	Last 30 days transaction records	Spending patterns, anomalies, and waste areas
Budget Recommendation	Budget limit and current spending	Personalized saving strategies and advice
Predictive Forecast	Three months transaction history	Predicted expenses by category and total forecast

8.5 Benefits of AI Integration

Without AI:

- Users manually analyze financial data.
- Budget tracking remains static.
- Notifications are generic.
- No future expense prediction capability.

With AI:

- Automatic detection of spending patterns.
- Personalized financial recommendations.
- Context-aware budget alerts.
- Predictive expense forecasting.
- Improved financial decision-making and user engagement.

9: References:

1. Google Gemini API Documentation. (2026). Gemini API Overview. https://ai.google.dev/gemini-api/docs
2. Flutter Documentation. (2026). Flutter SDK. https://flutter.dev/docs
3. Express.js. (2026). Express.js API Reference. https://expressjs.com
4. MongoDB. (2026). MongoDB Atlas Documentation. https://www.mongodb.com/docs/atlas/
5. JWT.io. (2026). JSON Web Tokens. https://jwt.io
6. Node.js. (2026). Node.js v20 Documentation. https://nodejs.org/docs